

CSE 3172: Mobile Application Development Lab

Credits	1
Contact Hours	28
Year	Third
Semester	First
Prerequisite	CSE 1222: Object Oriented Programming Lab, CSE 2232: User Interface Development Lab
Course Type	☐ Theory ☒ Laboratory work ☐ Project work ☐ Viva Voce

Motivation

Mobile applications are a primary mode of software use today. This course introduces React Native so students can extend their React.js knowledge to build cross-platform mobile apps with native UI, device APIs, and mobile deployment workflows.

Course Objective

This laboratory course introduces cross-platform mobile application development using React Native and Expo. Students will build multi-screen apps, use device APIs, manage local data, and produce a distributable build using EAS.

Course Outcomes (COs), Program Outcomes (POs) and Assessment

CO No.	CO Statement	Corresponding PO	Domain / Level of Learning Taxonomy	Delivery Methods & Activities	Assessment Tools
CO1	To set up a React Native / Expo project and apply React.js knowledge to mobile UI development	Modern Tool Usage (PO5)	Cognitive domain – Level 3	☐ Lecture Note ☒ Text Book ☐ Audio/Video ☒ Web Material ☒ Lab Manual	☒ CA ☐ Final Exam ☒ Assignment ☒ Notebook ☒ Presentation
CO2	To build multi-screen mobile applications using React Navigation and device APIs	Modern Tool Usage (PO5)	Cognitive domain – Level 6	☐ Lecture Note ☒ Text Book ☐ Audio/Video ☒ Web Material ☒ Lab Manual	☒ CA ☐ Final Exam ☒ Assignment ☒ Notebook ☒ Presentation
CO3	To develop a data-driven React Native application using the Fetch API and EAS Build	Design/Development of Solutions (PO3)	Cognitive domain – Level 6	☐ Lecture Note ☒ Text Book ☐ Audio/Video ☒ Web Material ☒ Lab Manual	☒ CA ☒ Final Exam ☒ Assignment ☒ Notebook ☒ Presentation

Assessment and Marks Distribution

Component	Weightage
Continuous Assessments (CA) — lab exercises and assignments throughout the semester	20%
Comprehensive Final Exam + Lab Notebook	70%
Class Participation	10%

Lab Course Contents / List of Experiments

Lab 1 — Environment Setup and React Native Fundamentals

- Install Node.js, Expo CLI (`npm install -g expo-cli`), and the **Expo Go** app on a physical device
- Create a new project: `npx create-expo-app MealExplorer`
- Project structure: `app.json` , `App.js` , `assets/` , `node_modules/`
- React Native vs. React web — key differences:

React (Web)	React Native
<code><div></code>	<code><View></code>
<code><p></code> , <code></code>	<code><Text></code>
<code></code>	<code><Image></code>
CSS files / Tailwind	<code>StyleSheet.create({})</code>
<code>onClick</code>	<code>onPress</code>

- Run on device via Expo Go (scan QR code) and on emulator (Android Studio AVD / iOS Simulator)
- Hot reload and fast refresh workflow

Lab 2 — Core Components and Layouts

- `<View>` , `<Text>` , `<Image>` , `<TextInput>` , `<Button>` , `<Pressable>` , `<TouchableOpacity>`
- Scrollable content: `<ScrollView>` vs. `<FlatList>` (performance difference, `keyExtractor` , `renderItem`)
- `<SafeAreaView>` and handling notches/status bars
- `<StatusBar>` customisation

Lab 3 — Styling and Flexbox in React Native

- `StyleSheet.create()` : why it is preferred over inline styles
- React Native Flexbox: `flexDirection` defaults to `column` (unlike CSS); `justifyContent` , `alignItems` , `flex`
- Dimensions API: `Dimensions.get('window')` for responsive sizing
- Platform-specific styles: `Platform.OS` , `Platform.select({})`

- Styling exercise: recreate a mobile card layout matching a provided Figma mockup

Lab 4 — Navigation with React Navigation

- Install React Navigation: `@react-navigation/native`, `@react-navigation/stack`, `@react-navigation/bottom-tabs`
- **Stack Navigator:** pushing and popping screens, passing params (`route.params`), header customisation
- **Bottom Tab Navigator:** tab icons with `@expo/vector-icons`, active tab styling
- Combining navigators: a tab navigator where one tab contains a stack
- Navigation exercise: build a 3-screen app (Home → List → Detail) with back navigation

Lab 5 — User Input, Forms, and Keyboard Handling

- Controlled `<TextInput>`: `value`, `onChangeText`, `secureTextEntry`, `keyboardType`
- `KeyboardAvoidingView` and `ScrollView` to prevent keyboard overlap
- Basic form validation in React Native (required fields, email format)
- `Alert.alert()` for native confirmation dialogs
- `TouchableOpacity` vs. `Pressable`: pressed state feedback

Lab 6 — Fetching Data and Async State

- `useEffect` + `fetch` in React Native (same pattern as CSE 2232 web apps)
- `ActivityIndicator` for loading states; conditional rendering for errors and empty states
- `FlatList` with fetched data: `refreshing` prop + pull-to-refresh (`onRefresh`)
- Caching strategy: storing last-fetched result in `useState` to avoid redundant calls
- Practice: fetch and display a list from [REST Countries API](#) with a search filter

Lab 7 — Device APIs: Camera and Location

- **Camera / Image Picker:** `expo-image-picker` — request permissions, pick from gallery, capture photo, display result in `<Image>`
- **Location:** `expo-location` — request foreground permission, `getCurrentPositionAsync()`, display coordinates
- **Permissions model:** `PermissionsAndroid` vs. Expo permissions API; why permissions must be requested at runtime
- **Linking:** `Linking.openURL()` to open maps, phone dialler, or external browser from within the app
- Exercise: build a "Location Card" screen that shows current coordinates and a static map link

Lab 8 — Notifications and Local Storage

- **Push Notifications:** `expo-notifications` — scheduling a local notification, notification channels (Android), foreground notification handler
- **AsyncStorage:** `@react-native-async-storage/async-storage` — `setItem`, `getItem`, `removeItem` for persisting lightweight data (e.g., user favourites, last-visited screen)
- Exercise: add a "Save to Favourites" feature using `AsyncStorage`; show saved items on a dedicated Favourites screen

Lab 9 — EAS Build: Producing a Distributable App

- What EAS (Expo Application Services) is and why it replaces `expo build`
- Install and configure EAS CLI: `npm install -g eas-cli` → `eas login` → `eas build:configure`
- `app.json` / `eas.json` configuration: `bundleIdentifier` (iOS), `package` (Android), build profiles (`development`, `preview`, `production`)
- Trigger an **APK build** for Android: `eas build --platform android --profile preview`
- Download and install the `.apk` on a physical Android device
- Overview of signing (keystore for Android, provisioning profile for iOS) — concepts only, not hands-on for iOS
- Exercise: produce a working `.apk` of the Lab 8 exercise app and install it on a classmate's device

Lab 10 — React Native Capstone: Meal Explorer Mobile App (Final Project)

Build a complete mobile version of the Meal Explorer application built in CSE 2232, using [TheMealDB API](#) (free, no auth required).

App screens:

Screen	API Endpoint	Key Feature
Categories Screen	<code>categories.php</code>	<code>FlatList</code> of category cards (thumbnail + name); tap to navigate
Meals by Category Screen	<code>filter.php?c={category}</code>	Grid of meal thumbnails for the selected category
Meal Details Screen	<code>lookup.php?i={mealId}</code>	Full details — image, ingredients, step-by-step instructions, YouTube button (<code>Linking.openURL</code>)
Favourites Screen	—	Meals saved via <code>AsyncStorage</code> ; accessible from the bottom tab bar

Technical requirements: - Bottom Tab Navigator (Categories, Favourites) with a Stack Navigator inside Categories for the drill-down flow - `ActivityIndicator` loading state and error message on every data-fetching screen - Pull-to-refresh on the Meals by Category screen - Save / unsave meals to `AsyncStorage` from the Details screen; reflect changes live on the Favourites tab - Responsive layout using React Native Flexbox; tested on both Android and iOS screen sizes - Accessible: all images have `accessibilityLabel`, interactive elements have `accessibilityRole`

EAS Build deliverable: - Configure `eas.json` with a `preview` profile - Run `eas build --platform android --profile preview` to produce an `.apk` - Submit the Expo build URL or the downloaded `.apk` file alongside the lab notebook

Assessment: Students demo the live app on a physical device or emulator (all four screens functional), present their EAS build artefact, and submit the complete lab notebook covering Labs 1–10.

Alternative Final Projects: Students may choose one of the following in place of the Meal Explorer. All alternatives mirror the same apps from CSE 2232 but rebuilt natively in React Native — reinforcing the "same logic, different platform" principle. All require the same four-screen structure (browse → filter → detail → favourites), Stack + Tab Navigation, `AsyncStorage`, and an EAS APK build.

Project	API	Auth Required	Mobile-Specific Feature vs. CSE 2232 Web Version
Country Explorer	REST Countries	No	Tap a bordering country on the Details screen to navigate directly to its detail; use <code>expo-location</code> to highlight the user's own country on launch
Movie Explorer	OMDb API	Free API key	Search-driven entry (TextInput + FlatList); save watchlist to AsyncStorage; open trailer via <code>Linking.openURL</code>
Music Explorer	iTunes Search API	No	30-second track preview using <code>expo-av</code> (<code>Audio.Sound</code>) — replaces the web <code><audio></code> tag with native audio playback

Regardless of project choice, all technical requirements above (Tab + Stack Navigator, `ActivityIndicator`, pull-to-refresh, AsyncStorage favourites, accessibility labels, EAS APK build) apply.

Textbook

1. Nader Dabit — *React Native in Action*, Manning Publications, 2019
2. Bonnie Eisenman — *Learning React Native* (2nd ed.), O'Reilly Media, 2018

Recommended Resources

1. **React Native Official Docs** — reactnative.dev — component API reference and guides
2. **Expo Documentation** — docs.expo.dev — Expo SDK APIs, EAS Build, and managed workflow
3. **React Navigation Docs** — reactnavigation.org — navigation patterns and API reference
4. **EAS Build Guide** — docs.expo.dev/build/introduction — configuring and running EAS builds

Free PDF Quick-Reference Books (goalkicker.com)

Topic	Download
React JS	React JS Notes for Professionals (PDF)
React Native	React Native Notes for Professionals (PDF)

Topic	Download
JavaScript	JavaScript Notes for Professionals (PDF)

React Native shares the same JavaScript and React fundamentals as CSE 2232. Students are encouraged to keep the React JS and JavaScript Goalkicker PDFs from that course as ongoing references.